Institución Universitaria ITM

Laboratorio de Optimización de Consultas

Diagnóstico, intervención y medición sobre el Mini-Lakehouse TuBoleta (DuckDB + Parquet)

Informe técnico — Taller 4

Curso: Tópicos Avanzados en Bases de Datos

Integrantes
Luis Manuel Contreras
Juan Pablo Salazar

Docente
Roberto Carlos Rahamut Suteu

Medellín, 3 de noviembre de 2026

Repositorio: github.com/JPS4L4/tuboleta_lakehouse

1. Resumen ejecutivo

Este taller partió del sistema analítico que construimos en el Taller 3: la información de venta de boletas de TuBoleta para el Estadio Atanasio Girardot, organizada en una zona "Gold" de archivos Parquet que consultamos con DuckDB. Elegimos tres preguntas de negocio representativas: **¿cuánto vendió cada evento en septiembre?** (Q1), **¿cuánto vendió cada zona del estadio en septiembre?** (Q2) y **¿cuántos intentos de compra terminaron en compra, abandono o rechazo?** (Q3).

En lugar de suponer por qué una consulta es lenta, le pedimos al motor que nos mostrara su plan de ejecución, es decir, el paso a paso de cómo resuelve cada pregunta. Encontramos lo siguiente:

- **Q1 y Q2 leían todos los datos de ventas para responder sobre un solo mes.** El archivo estaba guardado como un único bloque sin ordenar, así que el motor no tenía forma de saltarse los meses que no interesaban.
- **Q3 no tenía ningún problema.** Necesita mirar todos los intentos de compra, y eso es justo lo que hacía.

Aplicamos una corrección por consulta: ordenar el archivo por fecha en bloques pequeños (Q1), separar los datos en carpetas por mes (Q2) y actualizar las estadísticas del motor (Q3, como experimento de control). Las dos primeras funcionaron en lo que controlan: **el motor pasó a leer 4,5 y 10 veces menos datos**. Pero el tiempo casi no cambió: Q1 mejoró un 5 %, Q2 empeoró un 9 % y Q3 quedó igual.

Lo más valioso que aprendimos es por qué. Con unas 200.000 ventas, cada consulta tarda entre 2 y 5 milisegundos, y la mitad de ese tiempo es un costo fijo del motor (preparar la consulta, abrir archivos) que no depende de cuántos datos se lean. Las optimizaciones de acceso atacan la otra mitad. A esta escala el beneficio existe y es medible, pero el tamaño de los datos lo limita. El aprendizaje para el diseño es que estas decisiones son baratas de tomar al construir el pipeline y su beneficio crece con el volumen. Por eso conviene incorporarlas desde el inicio, sin esperar que aceleren un sistema pequeño.

2. Diagnóstico

Entorno de medición. DuckDB 2.0.0 sobre el Gold en Parquet del M3, sin modificar · AMD Ryzen 5 9600X (12 hilos lógicos) · 15,1 GB RAM · Windows 11 Pro · Python 3.14.6. Cada tiempo es la mediana de 5 ejecuciones, después de 3 ejecuciones de calentamiento. Los planes completos están en `docs/taller4/planes/` y se regeneran con `python -m scripts.taller4_optimizacion`.

Una particularidad de DuckDB 2.0. El árbol de `EXPLAIN ANALYZE` informa filas reales, archivos y row groups, pero muestra `0µs` en todos los operadores. Para nombrar el operador más costoso con evidencia, el script captura además el profiling JSON del motor (`profiling_mode='detailed'`), que trae el tiempo y la cardinalidad estimada de cada operador. Ese tiempo es **CPU sumada entre hilos**, no tiempo de reloj, así que se usa para comparar operadores entre sí.

2.1 Tabla de diagnóstico

Campo	Q1 — Ingresos por evento, sep-2026	Q2 — Ingresos por zona, sep-2026	Q3 — Embudo de conversión
Tiempo baseline (mediana)	5,09 ms	2,31 ms	4,98 ms
Motor usado	DuckDB 2.0.0 / Parquet	DuckDB 2.0.0 / Parquet	DuckDB 2.0.0 / Parquet
Operador más costoso	`TABLE_SCAN` de la fact: 1,10 ms de CPU (42 %), `Row Groups Scanned: 1 / 1`, 640,9 kB	`TABLE_SCAN` de la fact: 1,04 ms de CPU (86 %), `1 / 1`, 583,7 kB	`TABLE_SCAN` de intentos: 1,23 ms de CPU (43 %), 260.294 filas; `HASH_GROUP_BY` 0,98 ms
Problema identificado	Acceso secuencial sin data skipping posible: 1 row group, fact sin ordenar por `tiempo_sk`	Acceso secuencial de un archivo único para un filtro que retiene el 16,3 % de las filas	Ninguno de acceso: agregación total sin filtro
Indicador que lo revela	1 (208.236 filas leídas, 2 devueltas) → causa raíz 4	1 → causa raíz 4	Ninguno
Indicadores descartados	3 (dos `HASH_JOIN`, build en dims de 720 y 5 filas) · 5 (`mes INTEGER = 9`, sin `CAST`) · 2 presente pero inocuo (ver abajo)	2 (estimadas 41.647 frente a 34.039 reales, 1,2x) · 3 (sin joins) · 5 (`tiempo_sk BIGINT` contra enteros)	2 (estimaciones exactas) · 3 (`HASH_JOIN`, sin `CROSS_JOIN`) · 4 y 5 (sin predicado)

2.2 Q1 — Ingresos por evento en septiembre 2026

```
SELECT e.nombre_evento, t.mes, COUNT(*) AS boletas, SUM(f.precio_pagado) AS ingresos
FROM fact_ventas_boletas f
JOIN dim_tiempo t ON t.tiempo_sk = f.tiempo_sk
JOIN dim_evento e ON e.evento_sk = f.evento_sk
WHERE t.anio = 2026 AND t.mes = 9
GROUP BY 1, 2
```

Fragmento del plan (`Q1_antes.txt`), leído de abajo hacia arriba:

```
Summary:  Total Time: 0.0079s · Data Read: 640.9 kB
Hash Join (INNER)  tiempo_sk = tiempo_sk                          34,039 rows
├─ Table Scan  fact_ventas_boletas.parquet                         34,039 rows
│    Dynamic Filters: optional: tiempo_sk IN PRF(tiempo_sk)
│    Row Groups Scanned: 1 / 1
└─ Filter  tiempo_sk BETWEEN 2026022310 AND 2026101623                720 rows
   └─ Table Scan  dim_tiempo.parquet   Filters: mes = 9               720 rows
```

Lectura. DuckDB filtra `dim_tiempo` por `mes = 9` y obtiene 720 filas, las horas de septiembre. A esa dimensión le aplica además un rango que coincide exactamente con el min/max de `tiempo_sk` en la fact: es un filtro derivado de las estadísticas del archivo. Con esas 720 filas construye la tabla hash del join, y de ella sale un **filtro dinámico** (`PRF`) que se empuja al scan de la fact. Ese filtro solo puede saltarse bloques si las estadísticas min/max de cada bloque (zonemaps) lo permiten. Como la fact tiene **un único row group** (`1 / 1`) cuyo rango cubre todo el período, no se puede descartar nada: se decodifica la tabla completa para quedarse con el 16,3 %.

Por qué es el Indicador 1 con causa en el 4. El motor lee 208.236 filas para devolver 2: es acceso secuencial. Lo que impide evitarlo es la ausencia de data skipping, porque la fact no está ordenada por `tiempo_sk` (`is_monotonic_increasing = False`) y se escribió sin `row_group_size`.

Hallazgo secundario: Indicador 2. El profiling muestra que la salida de los joins se estimó en **1.651 filas y fueron 34.039 (20,6x)**, por encima del umbral de 10x del enunciado. Lo documentamos, pero no es el problema principal: aun con esa estimación, el planner eligió el algoritmo correcto (Hash Join) y las tablas build correctas (las dimensiones pequeñas). Un error de estimación solo es un problema cuando lleva a una mala decisión, y aquí no la hubo.

2.3 Q2 — Ingresos por zona en septiembre 2026

```
SELECT zona_sk, COUNT(*) AS boletas, SUM(precio_pagado) AS ingresos
FROM fact_ventas_boletas
WHERE tiempo_sk BETWEEN 2026090100 AND 2026093023
GROUP BY 1
```

```
Summary:  Total Time: 0.0026s · Data Read: 583.6 kB
Perfect Hash Group By                                              4 rows
└─ Table Scan  fact_ventas_boletas.parquet                      34,039 rows
     Filters: (tiempo_sk >= 2026090100) AND (tiempo_sk <= 2026093023)
     Row Groups Scanned: 1 / 1
```

Lectura. Este es el caso puro del problema. El filtro está directo sobre la fact y DuckDB lo empuja al scan (no hay `CAST : BIGINT` contra literales enteros, así que se descarta el Indicador 5). La estimación es razonable (41.647 frente a 34.039). Aun así se leen los 583,7 kB de las tres columnas proyectadas, porque el único row group contiene todos los meses. El scan concentra el 86 % de la CPU de la consulta. DuckDB elige `PERFECT_HASH_GROUP_BY` porque conoce el rango pequeño de `zona_sk`: un detalle que volverá a importar en la intervención.

2.4 Q3 — Embudo de conversión por evento

```
SELECT d.nombre_evento, COUNT(*) AS intentos, SUM(i.es_compra) AS compras,
       SUM(i.es_abandono) AS abandonos, SUM(i.es_rechazo_unique) AS rechazos
FROM fact_intentos_compra i JOIN dim_evento d ON d.evento_sk = i.evento_sk
GROUP BY 1
```

```
Hash Group By                                                    5 rows
└─ Hash Join (INNER)  evento_sk = evento_sk                   260,294 rows
   ├─ Table Scan  fact_intentos_compra.parquet   1 / 1        260,294 rows
   └─ Table Scan  dim_evento.parquet                               5 rows
```

Lectura. Las estimaciones son exactas en todos los nodos (260.294 = 260.294 y 5 = 5), el join es un `HASH_JOIN` con build en la dimensión de 5 filas, y no hay `WHERE`. Que se lean 260.294 filas para devolver 5 **no** es el Indicador 1: esa reducción la hace la agregación, no un filtro, y la consulta necesita todas las filas. No existe un subconjunto que un índice, un zonemap o una partición pudieran evitar. **Diagnóstico: no hay problema que resolver.** Por eso Q3 se usa como experimento de control.

3. Intervenciones

Aplicamos una sola intervención por problema. El Gold original no se modifica: cada intervención escribe en `lakehouse/gold_t4/`, y el script verifica con `assert` que la consulta devuelve exactamente el mismo resultado antes y después. Para explicar los resultados modestos corrimos además controles de medición que **no son intervenciones**, en `scripts/taller4_complementario.py` (mediana de 15 corridas).

3.1 I-3 en Q1 — Ordenar la fact por `tiempo_sk` en row groups de 20.000 filas

```
df.sort_values("tiempo_sk").to_parquet(ordenado, index=False, row_group_size=20_000)
```

	Antes	Después
Row groups de la fact	1	11
`Row Groups Scanned`	1 / 1	**3 / 11**
`Data Read`	640,9 kB	**142,4 kB (4,5x menos)**
Operadores del plan	11 (2 `HASH_JOIN`, `HASH_GROUP_BY`)	los mismos 11
Tiempo (mediana)	5,09 ms	4,83 ms → **1,05x**

Por qué funciona. Parquet guarda min/max por columna en cada row group. Con la fact ordenada, cada row group cubre un rango contiguo de `tiempo_sk`, y el filtro dinámico que llega desde `dim_tiempo` se compara contra esos rangos. Revisando los metadatos del archivo, solo los row groups 7 (2026080615–2026090116), 8 (2026090116–2026091020) y 9 (2026091020–2026101211) intersectan septiembre: exactamente los 3 de 11 que reporta el plan. Se decodifican unas 60.000 filas en lugar de 208.236. Que los bytes bajen todavía más (4,5x) es consistente con que una columna ordenada se comprime mejor.

Por qué el tiempo mejora tan poco. Lo medimos en lugar de suponerlo:

Fact	Hilos	Mediana (ms)	CPU de operadores (ms)	Fuera de los operadores (ms)
original (1 row group)	1	4,80	2,40	2,40
original (1 row group)	12	5,23	2,73	—
ordenada (11 row groups)	1	**4,09**	**1,56**	2,35
ordenada (11 row groups)	12	4,70	5,75	—

1. **Overhead fijo.** Unos 2,4 ms por ejecución ocurren fuera de los operadores (planificación, apertura de archivos, metadatos). Es la mitad del tiempo, e I-3 no lo toca, así que el techo de mejora a esta escala ronda 2x.
2. **Paralelismo.** El row group es la unidad de paralelismo de DuckDB sobre Parquet. Con 1 row group, el scan es secuencial: la CPU casi no cambia entre 1 y 12 hilos. Con 11, DuckDB reparte el trabajo y luego combina agregaciones parciales, y esa coordinación hace subir la CPU de 1,56 a 5,75 ms. **Aislando el efecto con 1 hilo, I-3 reduce la CPU un 35 % y el tiempo 1,17x.**
3. **Caché.** Después del calentamiento el archivo está en la caché del sistema operativo: leer 500 kB menos ahorra decodificación, no I/O de disco.

La estimación del join (1.651 frente a 34.039) no cambia: reordenar los datos no le da al planner estadísticas nuevas sobre la relación entre las tablas.

3.2 I-6 en Q2 — Particionado Hive `year=/month=`

```
df["year"]  = df["tiempo_sk"] // 1_000_000       # la fact no tiene fecha: se deriva de YYYYMMDDHH
df["month"] = (df["tiempo_sk"] // 10_000) % 100
for (y, m), g in df.groupby(["year", "month"]):
    g.drop(columns=["year", "month"]).to_parquet(f"{base}/year={y}/month={m:02d}/data.parquet", index=False)
# consulta: read_parquet('.../**/*.parquet', hive_partitioning=true) WHERE year = 2026 AND month = 9
```

	Antes	Después
Archivos leídos	1 (toda la fact)	**1 de 9** (`Scanning Files: 1/9`)
`Data Read`	583,7 kB	**58,8 kB (9,9x menos)**
Filtro	`tiempo_sk` por rango, empujado al scan	`CAST(month AS INTEGER) = 9`, sobre la ruta
Filas estimadas / reales	41.647 / 34.039	**1** / 34.039
Agregación	`PERFECT_HASH_GROUP_BY`	`HASH_GROUP_BY`
Tiempo (mediana)	2,31 ms	2,55 ms → **0,91x (empeoró)**

Qué funcionó. El partition pruning se resuelve sobre los nombres de carpeta antes de abrir ningún Parquet: de 9 archivos solo se abre `month=09`, y los bytes leídos caen casi 10x. El `CAST` sobre `month` es técnicamente un Indicador 5, pero inofensivo: se evalúa sobre 9 valores de partición, no sobre 208.236 filas.

Por qué empeoró. Separamos el costo del mecanismo del costo de los datos con un control:

Variante (mediana de 15)	ms
Archivo único, filtro por rango (baseline)	2,26
Glob Hive + partition pruning (I-6)	2,52
Lectura directa de `month=09/data.parquet`	**1,08**

Leer solo septiembre cuesta la mitad que el baseline, así que la reducción de datos es real. Lo que se come la ganancia, unos 1,4 ms, es el mecanismo: expandir el glob `**/*.parquet`, listar 9 carpetas y parsear los valores `year=` / `month=` de cada ruta. Además, al leer por glob el planner **pierde las estadísticas**: estima 1 fila en lugar de 34.039 (la intervención introdujo un Indicador 2 que antes no existía) y abandona `PERFECT_HASH_GROUP_BY`, que necesita conocer el rango de `zona_sk`. Es el clásico problema de los archivos pequeños: el particionado conviene cuando cada partición es mucho más grande que el costo de listarla (millones de filas por mes, o almacenamiento de objetos donde cada archivo descartado ahorra una petición de red), no con 208.000 filas en 9 archivos.

3.3 I-2 en Q3 — `ANALYZE` (experimento de control)

Sobre vistas de Parquet `ANALYZE` no tiene estadísticas propias que actualizar, así que el control se hace sobre tablas materializadas en DuckDB. El plan y la mediana se toman antes y después de `ANALYZE` **en la misma conexión**, para que `ANALYZE` sea la única variable.

	Antes	Después
Plan	`HASH_GROUP_BY` ← `PROJECTION` ← `HASH_JOIN` ← 2 × `TABLE_SCAN`	idéntico (verificado por el script)
Estimaciones	260.294 / 5, exactas	260.294 / 5
Tiempo (mediana)	2,58 ms	2,43 ms → **1,06x (ruido)**

Justificación. `ANALYZE` corrige estadísticas desactualizadas o ausentes, y aquí las estimaciones ya eran exactas: el resultado nulo era la predicción, y confirmarlo valida el diagnóstico.

Experimento adicional. ¿`ANALYZE` habría corregido el error de 20,6x de Q1? Lo probamos con las estimaciones de `EXPLAIN`:

Fuente de datos	Estimada antes	Estimada después de `ANALYZE`	Real
Vistas sobre Parquet	1.651	1.651	34.039
Tablas materializadas	24.037	24.037	34.039

`ANALYZE` no mueve la estimación en ningún caso. Lo que la mejora, de 20,6x a 1,4x de error, es **materializar**: al cargar una tabla nativa, DuckDB recolecta estadísticas propias que una vista sobre Parquet no tiene. El Indicador 2 de Q1 nace de consultar Parquet a través de vistas, no de estadísticas viejas, y `ANALYZE` no es la herramienta que lo resuelve.

4. Tabla de benchmarks

Query	Problema (vocabulario M4)	Intervención	t antes (ms)	t después (ms)	Factor	¿Cambió el plan?
Q1 — Ingresos por evento, sep-2026	Acceso secuencial sin data skipping: 1 row group, fact sin ordenar; 208.236 filas leídas, 2 devueltas	I-3: ordenar por `tiempo_sk`, `row_group_size=20_000`	5,09	4,83	1,05x	Sí, en el acceso: row groups 1/1 → 3/11, 640,9 → 142,4 kB. Mismos operadores.
Q2 — Ingresos por zona, sep-2026	Acceso secuencial de un archivo único para un 16,3 % de filas	I-6: particionado Hive `year/month`	2,31	2,55	0,91x	Sí: archivos 1/9, 583,7 → 58,8 kB; `PERFECT_HASH` → `HASH_GROUP_BY`; estimación 41.647 → 1
Q3 — Embudo de conversión	Ninguno: agregación total, estimaciones exactas	I-2: `ANALYZE` (control)	2,58	2,43	1,06x	No: plan idéntico

Contexto de la medición. DuckDB 2.0.0 · Ryzen 5 9600X · 15,1 GB · Windows 11 Pro. Calentamiento ×3, mediana de 5. Q3 se mide sobre tablas materializadas (por eso su "antes" es menor que su baseline de diagnóstico sobre Parquet, 4,98 ms).

Variabilidad. En las 6 corridas completas que hicimos durante el desarrollo, el factor de Q1 osciló entre 1,05x y 1,20x, el de Q2 entre 0,87x y 1,23x, y el de Q3 entre 0,81x y 1,06x. Con consultas de 2 a 5 ms, una diferencia de ±0,3 ms entre corridas ya es ruido. Por eso las conclusiones se apoyan en lo que el plan muestra de forma determinista (row groups, archivos y bytes leídos, operadores elegidos) y en controles con 15 corridas, no en un solo factor de tiempo.

Lectura honesta. Las dos intervenciones con diagnóstico real hicieron lo que la teoría predice sobre el acceso: 4,5x y 9,9x menos datos leídos. Ninguna se tradujo en una mejora de tiempo significativa, por razones identificadas y medidas: el overhead fijo, el costo de coordinar el paralelismo y el costo del glob y las rutas Hive. El control (Q3) dio el resultado nulo esperado.

5. Reflexión arquitectural

Si pudiéramos rediseñar el pipeline del M3 sabiendo lo que sabemos ahora, ¿qué cambiaríamos?

1. El orden físico del Gold es una decisión de diseño, no una optimización posterior. `transform_gold.py` escribe cada tabla con `df.to_parquet(ruta, index=False)`, sin ordenamiento ni `row_group_size`. Eso deja la fact en un único row group sin orden, lo que anula a la vez el data skipping y el paralelismo del scan. Ordenar por `tiempo_sk` y fijar el tamaño de row group cuesta un `sort_values` en una carga que ya tarda 1 segundo, y su beneficio crece con el volumen. Lo incorporaríamos al pipeline desde el inicio. El tamaño de row group, eso sí, debe elegirse para el volumen real: 20.000 filas sirvió para demostrar el mecanismo, pero a esta escala el paralelismo que habilita cuesta más de lo que ahorra. El valor del enunciado (100.000) o el que DuckDB usa por defecto (unas 122.000 filas) es más razonable cuando la fact crezca a millones.

2. El modelo dimensional ya tenía la herramienta, pero las consultas no la usaban. `tiempo_sk = YYYYMMDDHH` es un entero que preserva el orden cronológico, así que permite filtrar rangos de fecha directamente en la fact (como Q2) sin pasar por `dim_tiempo`. Aun así, las consultas que escribimos en el M3 filtran a través del join (como Q1). DuckDB compensa en parte con filtros dinámicos, pero depende de una optimización del motor. Un filtro directo sobre la clave es explícito y portable a otros motores. En un motor con índices (PostgreSQL o CockroachDB, nuestro OLTP de talleres anteriores), `tiempo_sk` sería la clave foránea candidata a un índice, y su formato ordenado la hace ideal para un índice B-tree de rango.

3. Nuestras consultas no siguen la clave de ordenamiento. De las 9 consultas de `analyze.py`, solo una tiene `WHERE` (y filtra por zona, no por tiempo). Las demás agregan la tabla completa con `FILTER` dentro del `SELECT`. Para ese patrón, el data skipping por fecha no aporta nada, porque no hay ningún bloque que se pueda saltar. La clave de ordenamiento y la de partición deben elegirse por los filtros reales de las consultas. Si el uso real es un tablero mensual, `tiempo_sk` es la clave correcta. Si es un reporte histórico completo, el beneficio vendría de otra técnica: pre-agregaciones.

4. Particionar no es gratis, y a nuestra escala es contraproducente. I-6 demostró que el partition pruning funciona, pero también que cada archivo tiene un costo fijo y que el planner pierde estadísticas al leer por glob. Con 208.000 filas, un archivo bien ordenado es mejor diseño que 9 carpetas. Particionaríamos solo cuando cada mes tenga millones de filas, o si el Gold viviera en almacenamiento de objetos.

5. Hay costos que ningún índice resuelve. La consulta 7 del M3 (`quantile_cont` sobre `tiempo_hasta_pago_seg`) y Q3 cuestan por la agregación, no por el acceso: deben recorrer todos los valores. En producción eso se resuelve con **vistas materializadas o tablas de pre-agregación** (por ejemplo, ventas por evento × zona × día), recalculadas en cada carga del pipeline. El hallazgo de I-2 refuerza esa idea: las tablas materializadas no solo evitan decodificar Parquet, también le dan al planner mejores estadísticas (la estimación del join de Q1 pasó de 20,6x a 1,4x de error).

6. Escala. El README del M3 ya decía que la diferencia se vuelve decisiva a millones de filas, y este taller lo confirma con números: el overhead fijo (unos 2,4 ms) es la mitad del tiempo de cada consulta, y solo deja de dominar cuando el volumen de datos lo supera. El siguiente experimento natural es subir `N_USUARIOS` en `scripts/config.py` (hoy 150.000) para encontrar el volumen en el que I-3 e I-6 se vuelven decisivas.

6. Anexos

A. Planes `EXPLAIN ANALYZE` completos. Están en el repositorio, en `docs/taller4/planes/`, porque sus árboles miden más de 200 columnas y no caben legibles en una página:

Archivo	Contenido
`Q1_antes.txt` / `Q1_despues.txt`	Q1 sobre la fact original y sobre la ordenada (I-3)
`Q2_antes.txt` / `Q2_despues.txt`	Q2 sobre el archivo único y sobre la versión particionada (I-6)
`Q3_antes.txt`	Q3 sobre vistas de Parquet (diagnóstico)
`Q3_antes_analyze.txt` / `Q3_despues.txt`	Q3 sobre tablas materializadas, antes y después de `ANALYZE` (I-2)
`*_perfil.txt`	Tiempo de CPU y filas reales y estimadas por operador (profiling JSON de DuckDB)

B. Documentos de trabajo. `docs/taller4/diagnostico_baseline.md` (entregable del diagnóstico), `intervenciones.md` (fichas completas), `benchmarks.md` y `analisis_complementario.md` (tablas generadas por los scripts).

C. Cómo reproducir.

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
python -m scripts.taller4_optimizacion     # genera el Gold del M3 si falta; planes, perfiles y benchmarks
python -m scripts.taller4_complementario   # controles de paralelismo, Hive y ANALYZE
```

Los tiempos absolutos varían según la máquina y la corrida. Lo determinista (row groups, archivos, bytes leídos, operadores y estimaciones) es reproducible con la semilla 42 del M3.